自主學習 專案製作

物理引擎：
圓與凸多邊形的剛體碰撞模擬

建國中學 陳顥恩

中華民國一一二年四月十日

摘要

本專題的目標是使用 C++ 及繪圖函式庫從零創造出一能模擬圓與凸多邊形碰撞的物理引擎，並具有一定的效能及真實度。透過建構基礎數學工具、引擎所需的物件類別，及實作模擬碰撞所需的演算法，並解決實踐中遇到的困難及修正錯誤，最終成功達成所設定的目標。雖在精確度及效能上有些許改進空間，但整體的真實度、可發展性都相當不錯，能在此基礎上進行大幅度的改進，如增加使用者介面、優化效能等。

目錄

1 動機 **1**

2 使用環境 **1**

3 數學工具 **1**

3.1 平面向量 1

3.2 多邊形相關 2

4 物件類別 **4**

4.1 基底類別：`Shape` 4

4.1.1 成員變數 4

4.1.2 成員函式 4

4.2 圓：`Circle` 4

4.2.1 成員變數 5

4.2.2 成員函式 5

4.3 多邊形：`Polygon` 5

4.3.1 成員變數 5

4.3.2 成員函式 5

5 碰撞檢測 **5**

5.1 圓和圓之間 6

5.2 多邊形和多邊形之間 6

5.2.1 分離軸定理 SAT 6

5.2.2 尋找接觸點、碰撞法向量、碰撞深度 8

5.3 圓和多邊形之間 9

5.3.1 碰撞與否 9

5.3.2 尋找接觸點、碰撞法向量、碰撞深度 10

6 碰撞反應 **12**

6.1 線性碰撞的初步嘗試：彈簧法 12

6.1.1 理論推導 12

6.1.2 實作 12

6.1.3 成果 12

6.2 線性碰撞的改進：衝量法 14

6.2.1 理論推導 14

6.2.2 實作 15

6.2.3 成果 15

6.3 引入旋轉，大功告成：衝量法 16
6.3.1 理論推導 16
6.3.2 實作 17

7 問題解決 **18**
7.1 穿模 18
7.2 能量不守恆 18

8 未來展望 **19**
8.1 新增使用者介面 19
8.2 碰撞偵測的效能改進 19
8.3 新增摩擦力 19

9 心得與收穫 **19**

10 附錄：作品連結 **20**
10.1 Github 連結 20
10.2 成果截圖 20
10.3 測試影片 20

1 動機

物理課本中，教授了能預測、計算一維彈性碰撞的公式。然而，我學習後卻感到不滿足：即使我能運用幾何技巧將其擴展至部份二維情況，但若是形狀複雜的物體呢？如果物體會旋轉呢？又，既然計算的 SOP 如此明確，又為何不將其交給擅長運算的電腦去做呢？

於是，我萌生了創造一個物理引擎的想法。在高一課程中有一門多元選修課，其中我學到利用 vpython 進行基本物理現象模擬的方法，並在期末作業中成功模擬了圓形駐波，可謂是嘗到了甜頭。但我仍舊不滿足，不滿足於僅能模擬圓形粒子、不滿足於用虎克定律模擬分明能用公式一次計算的碰撞.....。最終，我決定自己創造一個屬於我的物理引擎，不但能滿足求知慾，同時也作為專案製作的練習。

2 使用環境

使用 C++ 作為基底程式語言，並以 SFML[1]為繪圖函式庫。同時，使用 SFML 提供的向量類別 sf::Vector2f 進行向量運算。

3 數學工具

C++ 及 SFML 函式庫中均無提供向量運算相關的函式，有必要自行建構出所需的數學工具。本專案中，自行宣告的數學工具函式均在命名空間 `sfv` 內。

3.1 平面向量

- 浮點數相等 (`flt_equal`)：輸入兩向量 $\vec{v} = (v_x, v_y), \vec{u} = (u_x, u_y)$，若 $|v_x - u_x|$ 及 $|v_y - u_y|$ 均小於 ϵ 時 (ϵ 很小)，則視為兩向量相等並回傳 $true$；反之則回傳 $false$。
- 絕對值 (`abs`)：輸入 $\vec{v}$，回傳 $|\vec{v}|$。
- 平方 (`square`)：輸入 $\vec{v}$，回傳 $|\vec{v}|^2$。
- 行列式 (`det`)：輸入 $\vec{u}$ 及 $\vec{v}$，回傳 $\begin{vmatrix} \vec{u} \\ \vec{v} \end{vmatrix}$。
- 單位向量 (`unitv`)：輸入 $\vec{v}$，回傳 $\hat{v}$。
- 法向量 (`normalv`)：輸入向量，回傳其逆時針方向、與其垂直、長度相同的向量。
- 內積 (`dot`)：輸入 $\vec{u}$ 及 $\vec{v}$，回傳 $\vec{u} \cdot \vec{v}$。
- 距離 (`distance`)：輸入兩座標點 A, B，回傳 $\overline{AB}$。

[1]SFML 是一個跨平台的軟體開發函式庫，旨在為多媒體提供簡單的應用程式介面。詳見https://www.sfml-dev.org/documentation/2.5.1/

- 投影 (`projection_v`)：輸入 $\vec{u}$ 及 $\vec{v}$，回傳 $\vec{u}$ 在 $\vec{v}$ 上的正射影 $\vec{p}$：

$$\vec{p} = \frac{\vec{u} \cdot \vec{v}}{|\vec{v}|^2}\vec{v}$$

- 點線距 (`point_lineseg_dis`)：輸入點 P, A, B，回傳點 P 到線段 AB 的距離 d：

$$Let\ \overrightarrow{AP'} = \frac{\overrightarrow{AP} \cdot \overrightarrow{AB}}{\left|\overrightarrow{AB}\right|^2}\overrightarrow{AB}$$

$$d = \begin{cases} \left|\overrightarrow{PP'}\right| & ,if\ \ 0 \leq \overrightarrow{AP'} \cdot \overrightarrow{AB} \leq \left|\overrightarrow{AB}\right| \\ \\ min(\left|\overrightarrow{AP}\right|, \left|\overrightarrow{BP}\right|) & ,otherwise \end{cases}$$

- 線性變換-旋轉 (`rotate_trans`)：輸入向量 $\vec{v}$ 及角度 θ，輸出向量 $\vec{v'}$：

$$\vec{v'} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \vec{v}$$

3.2 多邊形相關

- 多邊形的質心 (`com_of_polygon`)：依照順時針順序輸入凸多邊形的頂點位置，回傳其質心位置 (假設其質量分布均勻)。

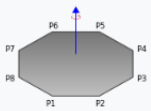
  作法：將 n 邊形分割為 $n-2$ 個三角形 (擇一頂點，對相鄰以外的頂點連線)，分別計算出所有三角形的質心，再以其面積為權重做加權平均。

  時間複雜度：$O(N)$，N 為多邊形的頂點數。

```
sf::Vector2f com_of_polygon(std::vector<sf::Vector2f> point_pos) {
    sf::Vector2f sum_v(0.0, 0.0);
    float sum_a = 0.0;
    for (int i = 0; i < point_pos.size() - 2; i++) {
        sf::Vector2f com_of_triangle = (point_pos[0] + point_pos[i +
           1] + point_pos[i + 2]) / 3.f;
        float area_of_triangle = det(point_pos[i + 1] - point_pos[0],
            point_pos[i + 2] - point_pos[0]) / 2.f;
        sum_v += com_of_triangle * area_of_triangle;
        sum_a += area_of_triangle;
        }
    return sum_v / sum_a;
}
```

- 多邊形的旋轉慣量 (`cacl_pol_inertia`)：輸入凸多邊形的頂點位置、多邊形的質量，回傳其旋轉慣量 (假設其質量分布均勻)。

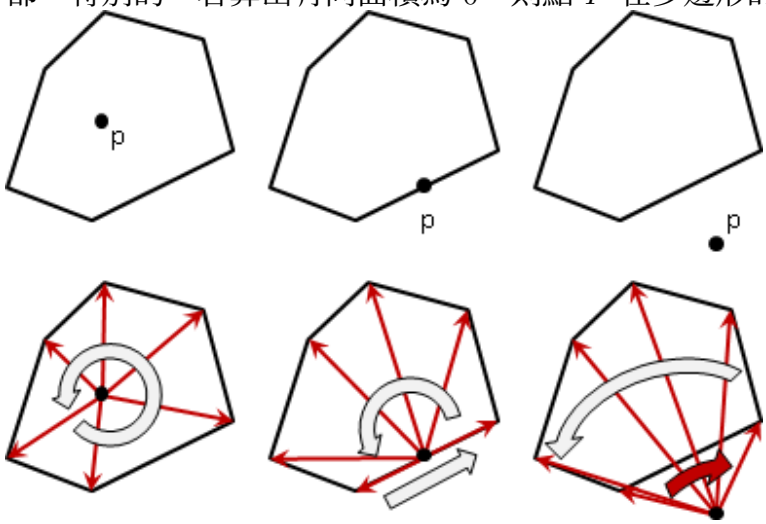
時間複雜度：$O(N)$，N 為多邊形的頂點數。

薄多邊形，頂點為$\vec{P}_1$，$\vec{P}_2$，$\vec{P}_3$，.....，$\vec{P}_N$，質量為m		$I = \frac{m}{6}\frac{\sum_{n=1}^{N}\lVert\vec{P}_{n+1}\times\vec{P}_n\rVert(\vec{P}_{n+1}^2+\vec{P}_{n+1}\cdot\vec{P}_n+\vec{P}_n^2)}{\sum_{n=1}^{N}\lVert\vec{P}_{n+1}\times\vec{P}_n\rVert}$

```
float cacl_pol_interia(float mass, std::vector<sf::Vector2f> points) {
    float numerator = 0, denominator = 0;
    for (auto iter = points.begin(); iter != points.end(); iter++) {
        auto next = iter;
        if (iter == points.end() - 1)
            next = points.begin();
        else
            next = iter + 1;
        numerator += sfv::cross(*iter, *next) * (sfv::dot(*iter, *iter) + sfv::dot(*iter, *next) + sfv::dot(*next, *next));
        denominator += sfv::cross(*iter, *next);
    }
    return (mass * numerator) / (6 * denominator);
}
```

- 判斷點在多邊形內 (point_in_polygon)：輸入一點 P、凸多邊形的頂點位置 $P_1, P_2, ..., P_N$，如果點在多邊形內則回傳 $true$；否則回傳 $false$。

  作法：利用行列式算出有向面積，判斷 $\overrightarrow{PP_1}, \overrightarrow{PP_2}, ..., \overrightarrow{PP_N}, \overrightarrow{PP_1}$ 是否均向同一個方向旋轉。若旋轉方向一致，則點在多邊形內部；若途中出現反向旋轉，則點在多邊形外部。特別的，若算出有向面積為 0，則點 P 在多邊形的邊上。



時間複雜度：$O(N)$，N 為多邊形的頂點數。

```
bool point_in_polygon(sf::Vector2f p, std::vector<sf::Vector2f> vertex) {
    float last = 0, now = 0;
    for (auto iter = vertex.begin(); iter != vertex.end(); iter++) {
        auto next = iter;
```

```
        if (iter == vertex.end() - 1)
            next = vertex.begin();
        else
            next = iter + 1;

        sf::Vector2f va = *iter - p, vb = *next - p;
        last = now;
        now = sfv::cross(va, vb);
        if (last == 0)
            continue;
        if (last * now > 0)
            continue;
        else
            return false;
    }
    return true;
}
```

4 物件類別

4.1 基底類別：Shape

Shape 作為圓、多邊形的母類別，須包含兩者皆具有的性質、功能。

4.1.1 成員變數

- 包含物體基本參數，如質量、轉動慣量、彈性係數。
- 物體隨時間而變的數值，如質心位置、速度、加速度、轉動角度、角速度。
- 物體的其他屬性，如顏色、是否不可移動。

4.1.2 成員函式

- 讀取、寫入成員變數。
- 物理操作：移動、轉動、受力。
- **步進：**
  給定一極小 Δt，對於每個時刻進行 $v = v_0 + a\Delta t$ 及 $x = x_0 + v\Delta t$ 運算，模擬物體的運動。

4.2 圓：Circle

為 Shape 的子類別，繼承其所有的成員變數及函式。

4.2.1 成員變數

- 圓的半徑。
- SFML 中的實體：`sf::CircleShape`。

4.2.2 成員函式

- 讀取、修改圓半徑。
- 回傳實體，供 SFML 函式繪圖用。
- update：將 SFML 實體之資料 (位置、半徑、顏色等) 與成員變數同步。

4.3 多邊形：Polygon

為 `Shape` 的子類別，繼承其所有的成員變數及函式。

4.3.1 成員變數

- 頂點數量。
- 頂點相對於質心的位置，以 `std::vector<float>` 進行儲存。
- SFML 中的實體：`sf::ConvexShape`。

4.3.2 成員函式

- 讀取頂點數量、位置。
- 回傳實體，供 SFML 函式繪圖用。
- update：將 SFML 實體之資料 (位置、半徑、顏色等) 與成員變數同步。

5 碰撞檢測

此步驟除了檢測兩物體是否碰撞，當碰撞時，還需計算接觸點位置、碰撞法向量、碰撞深度 (穿透量) 等資訊，以便日後在進行碰撞反應計算時使用。為此，這邊宣告了一個名為 `Collision_data` 的結構，作為碰撞檢測函式的回傳值。

```
#define INIT_VEC sf::Vector2f(FLT_MAX, FLT_MAX) // initial vector
#define INIT_SCA FLT_MAX    // initial scalar

struct Collision_data {
    bool collide = false;       // collide or not
    int contact_point_count = 1;
    std::vector<sf::Vector2f> normal = { INIT_VEC, INIT_VEC };
```

```
    std::vector<sf::Vector2f> contact_points = { INIT_VEC, INIT_VEC };
    float depth = INIT_SCA;
};
```

5.1 圓和圓之間

圓和圓的碰撞檢測是最簡單的。

兩圓之圓心位置分別為 C_1, C_2 且半徑分別為 r_1, r_2，若 $\overline{C_1C_2} <= r_1 + r_2$，則兩圓相碰撞。接觸點唯一，對圓心使用分點公式即可求出。碰撞法向量為 $\overrightarrow{C_1C_2}$，碰撞深度為 $(r_1 + r_2) - \overline{C_1C_2}$。

時間複雜度：$O(1)$

```
Collision_data if_collide(Circle& a, Circle& b) {
    Collision_data cd;

    cd.normal[0] = b.get_pos() - a.get_pos();
    cd.depth = a.get_radius() + b.get_radius() - sfv::abs(cd.normal[0]);
    cd.contact_points[0] = (a.get_pos() * b.get_radius() + b.get_pos() *
       a.get_radius()) / (a.get_radius() + b.get_radius());

    if (sfv::abs(a.get_pos() - b.get_pos()) < a.get_radius() + b.
       get_radius()) {
        cd.collide = true;
        return cd;
    }
    return cd;
}
```

5.2 多邊形和多邊形之間

多邊形間的碰撞情況較圓形複雜許多，需借助額外演算法。此處使用分離軸定理 (Separating Axis Theorem, 簡稱 SAT) 進行碰撞檢測，再額外計算出其他碰撞數據。

5.2.1 分離軸定理 SAT

SAT 的基本想法是：若兩凸多邊形沒有相交，則必然能找到一個方向，從該方向看去能看到兩凸多邊形間的間隙。並且，若兩物體是相交的，則從所有方向看去都不可能看到間隙。因此，只要一看到間隙，便可斷言兩物體沒有相交、也不需要再計算下去。這點也給了 SAT 相當快的運算速度，畢竟在物理引擎中，物體的碰撞往往只有一瞬間，絕大部分時間中它們都是不相交的。

在實踐中，會選定數個直線作為軸，並將兩凸多邊形投影到一軸上。若兩投影沒有重疊，則找到分離軸，確定兩凸多邊形不相交；反之則換一軸進行投影，如此重複進行。得益於凸多邊形的性質，可選定它們的邊的法向量作為投影軸 (即沿著邊的方向看去)，若全部迭代完

均有重疊，則可判斷兩凸多邊形有相交。

時間複雜度：$O(M^2 + MN + N^2)$，其中 M 與 N 分別為兩多邊形的邊數。

```
Collision_data if_collide(Polygon& a, Polygon& b) {     //using SAT

Collision_data cd;

// aps : a's points, bps : b's points
std::vector<sf::Vector2f> aps = a.get_point_pos(), bps = b.get_point_pos
    ();

for(int cnt = 0; cnt < 2; cnt++) {

    //taking one polygon to make axises
    std::vector<sf::Vector2f> taking;
    if (cnt == 0)
        taking = aps;
    else
        taking = bps;

    // taking edges' normal as axis to project on
    for (auto point = taking.begin(); point != taking.end(); point++) {

        // caculating the normal vector of the edge to project on
        sf::Vector2f normal;
        if (point != taking.end() - 1)
            normal = sfv::normalv(*next(point) - *point);   // point_next
                - point_now = vec(edge)
        else
            normal = sfv::normalv(*taking.begin() - *point);    // the
                next edge of the last edge is the first one

        // doing projection
        float amax = -FLT_MAX, amin = FLT_MAX, bmax = -FLT_MAX, bmin =
            FLT_MAX; // of the projection of the points of a,b on the axis
        for (auto a_point = aps.begin(); a_point != aps.end(); a_point++)
            {     //projecting a's points
            float projection = sfv::projection_f(*a_point, normal);
            amax = std::max(projection, amax);
                amin = std::min(projection, amin);
        }
        for (auto b_point = bps.begin(); b_point != bps.end(); b_point++)
            {     //projecting b's points
                float projection = sfv::projection_f(*b_point, normal);
                bmax = std::max(projection, bmax);
                bmin = std::min(projection, bmin);
        }

        // checking if is overlaping
        if (amin > bmax || amax < bmin) {       // do not overlap
                cd.collide = false;
                cd.depth = INIT_SCA;
                cd.normal[0] = INIT_VEC;
                return cd;      // not colliding
```

```
        }
    }
}
cd.collide = true;      // overlaping on every axis --> colliding
```

5.2.2 尋找接觸點、碰撞法向量、碰撞深度

以上程式碼檢測了碰撞與否，接下來要處理其他碰撞資訊。

計算接觸點的方法是窮舉兩凸多邊形的所有頂點，並首先排除不在另一多邊形內的頂點。對於剩下的頂點，尋找離其最接近的邊，並記錄該頂點與該邊的距離，作為各頂點的深度。最後以深度最深的頂點作為接處點，並且其深度為碰撞深度、最近邊的法向量為碰撞法向量。若有兩深度差值極小，則記錄兩個接觸點 (兩邊平行碰撞時會出現兩個接觸點)。

時間複雜度：$O(MN)$，其中 M 與 N 分別為兩多邊形的邊數。

```
cd.depth = -FLT_MAX;

for (int cnt = 0; cnt < 2; cnt++) {
    if (cnt == 1)
        std::swap(aps, bps);

    for (auto vertex : bps) {   // iterating through vertices
        std::vector<sf::Vector2f> depths;

        if (!sfv::point_in_polygon(vertex, aps))
                continue;

        float now_depth = FLT_MAX;
        sf::Vector2f now_normal = INIT_VEC;
        for (auto iter = aps.begin(); iter != aps.end(); iter++) {
           // iterating through edges

            sf::Vector2f start, end;    // (end - start) is the edge
            if (iter != aps.end() - 1)
                end = *(iter + 1), start = *iter;
           else
                end = *aps.begin(), start = *iter;

            if (sfv::point_lineseg_dis(vertex, start, end) < now_depth) {
                now_depth = sfv::point_lineseg_dis(vertex, start, end);
                now_normal = end - start;
            }
        }

        if (sfv::flt_equal(now_depth, cd.depth)) {
            if (sfv::flt_equal(vertex, cd.contact_points[0]))
                continue;
            cd.contact_point_count = 2;
            cd.contact_points[1] = vertex;
            cd.normal[1] = sfv::normalv(now_normal);
        }
        else if (now_depth > cd.depth) {
```

```
            cd.depth = now_depth;
            cd.contact_points[0] = vertex;
            cd.contact_point_count = 1;
            cd.normal[0] = sfv::normalv(now_normal);
        }
    }
}

// last check
if (cd.contact_points[0] == INIT_VEC && cd.depth == INIT_SCA)
    cd.collide = false;

return cd;
}
```

5.3 圓和多邊形之間

5.3.1 碰撞與否

圓和多邊形之間的碰撞檢測與多邊形和多邊形之間相近，一樣是使用 SAT。唯一不同的，選定的投影軸除了多邊形之邊的法向量外，還多了一條：圓心到離圓最近之多邊形頂點的方向向量。

時間複雜度：$O(N^2)$，其中 N 為多邊形的邊數。

```
Collision_data if_collide(Polygon& p, Circle& c) {      //using SAT
Collision_data cd;

// pps : p's points
const std::vector<sf::Vector2f> &pps = p.get_point_pos();
// cc : center of circle
sf::Vector2f cc = c.get_pos();

// taking edges' normal as axis to project on
for (auto point = pps.begin(); point <= pps.end(); point++) {

    // caculating the normal vector of the edge to project on
    sf::Vector2f normal;
    if (point != pps.end()) {
        if (point != pps.end() - 1)
            normal = sfv::normalv(*next(point) - *point);       //
                point_next - point_now = vec(edge)
        else
            normal = sfv::normalv(*pps.begin() - *point);       // the
                next edge of the last edge is the first one
    }
    else {      // point == pps.end() --> already done every edge of the
        polygon
        float min_dist = FLT_MAX;
        sf::Vector2f np;
        for (auto p : pps) {    // determine the vertex that is the
            nearest to the circle
            if (sfv::distance(cc, p) < min_dist)
```

```
                np = p;
        }
        normal = np - cc;
    }

    // doing projection
    float pmax = -FLT_MAX, pmin = FLT_MAX, cmax, cmin;  // min/max of the projection of the points of a,b on the axis
    for (auto iter = pps.begin(); iter != pps.end(); iter++) {  // projecting the polygon
        float projection = sfv::projection_f(*iter, normal);
        pmax = std::max(projection, pmax);
        pmin = std::min(projection, pmin);
    }
    float ccp = sfv::projection_f(cc, normal);  // the projection of cc
    cmax = ccp + c.get_radius();
    cmin = ccp - c.get_radius();

    // checking if is overlaping
    if (pmin > cmax || pmax < cmin) {   // do not overlap
        cd.depth = INIT_SCA;
        cd.normal[0] = INIT_VEC;
        cd.collide = false;
        return cd;      // not colliding
    }
    if (point == pps.end())     // prevent point from iterate over pps.end()
        break;
}

// is colliding
cd.collide = true;
```

5.3.2 尋找接觸點、碰撞法向量、碰撞深度

接觸點可分為以下兩種：

- 在多邊形的頂點上

  作法：檢查多邊形各頂點，先排除在圓外者。再分別計算其餘頂點離圓心的距離，取最近者作為接觸點，深度為圓的半徑減掉該點到圓心的距離。

- 在多邊形的邊上

  作法：計算多邊形各邊到圓心的距離 (利用數學工具中的 `point_lineseg_dis`)，若均大於半徑，則沒有此類型的接觸點。反之，取最近者，再將圓心投影到該邊上 (若投影點在線段外，則算作未找到，因為此時會有在頂點上的接觸點被找到)，圓心的投影點即為接觸點。深度為圓的半徑減掉投影點到圓心的距離。

若兩種接觸點皆有找到，則取深度較深的作為最終接觸點，其深度為最終深度。

時間複雜度：$O(N)$，其中 N 為多邊形的邊數。

```
cd.depth = -FLT_MAX;

// when the contact point is on one of the edges of p
for (auto point = pps.begin(); point != pps.end(); point++) {
    // seeing an edge as a segment of a line
    sf::Vector2f a = *point;    // taking point a, b
    sf::Vector2f b;
    if (point == pps.end() - 1)
        b = *pps.begin();
    else
        b = *(point + 1);
    sf::Vector2f ab = b - a;

    if (sfv::point_lineseg_dis(cc, a, b) > c.get_radius())
        continue;

    float ccp = sfv::projection_f(cc - a, ab);  // the projection of cc
    sf::Vector2f ccpv = a + sfv::projection_v(cc - a, ab);
    if (0 < ccp && ccp < sfv::abs(ab) && sfv::abs(ccpv - cc) < c.
       get_radius()){ // the projevtion of cc is between a and b  &&  d(
       cc, edge) < r
        float depth = c.get_radius() - sfv::point_lineseg_dis(cc, a, b);
        if (depth > cd.depth) {
            cd.depth = depth;
            cd.contact_points[0] = ccpv;        // contact point found
        }
    }
}


// when the contact point is on one of the vertices of p
for (auto& point : pps) {
    if (sfv::distance(point, cc) > c.get_radius())
        continue;
    float depth = c.get_radius() - sfv::distance(point, cc);
    if (depth > cd.depth) {
        cd.depth = depth;
        cd.contact_points[0] = point;   // contact point found
    }
}

if (cd.depth == FLT_MAX || cd.contact_points[0] == INIT_VEC) {
    cd.normal[0] = INIT_VEC;
    cd.collide = false;
    return cd;  // not colliding
}
cd.normal[0] = cd.contact_points[0] - cc;

return cd;
```

```
}
```

6 碰撞反應

6.1 線性碰撞的初步嘗試：彈簧法

彈簧法顧名思義，是在兩物體接觸時，在其間放置一假想彈簧，並依據虎克定律向兩物體施力。優點是想法簡單、易實作，缺點是容易失真。

6.1.1 理論推導

給定兩圓心位置分別為 C_1, C_2，半徑 r_1, r_2，質量 m_1, m_2，受彈簧作用的順時加速度 a_1, a_2。假想彈簧的彈力常數為 k。

$$\vec{x} = (r_1 + r_2 - \overline{C_1C_2}) \cdot \overrightarrow{C_1C_2}$$

$$\vec{a}_1 = -\frac{k\vec{x}}{m_1},\ \vec{a}_2 = \frac{k\vec{x}}{m_2}$$

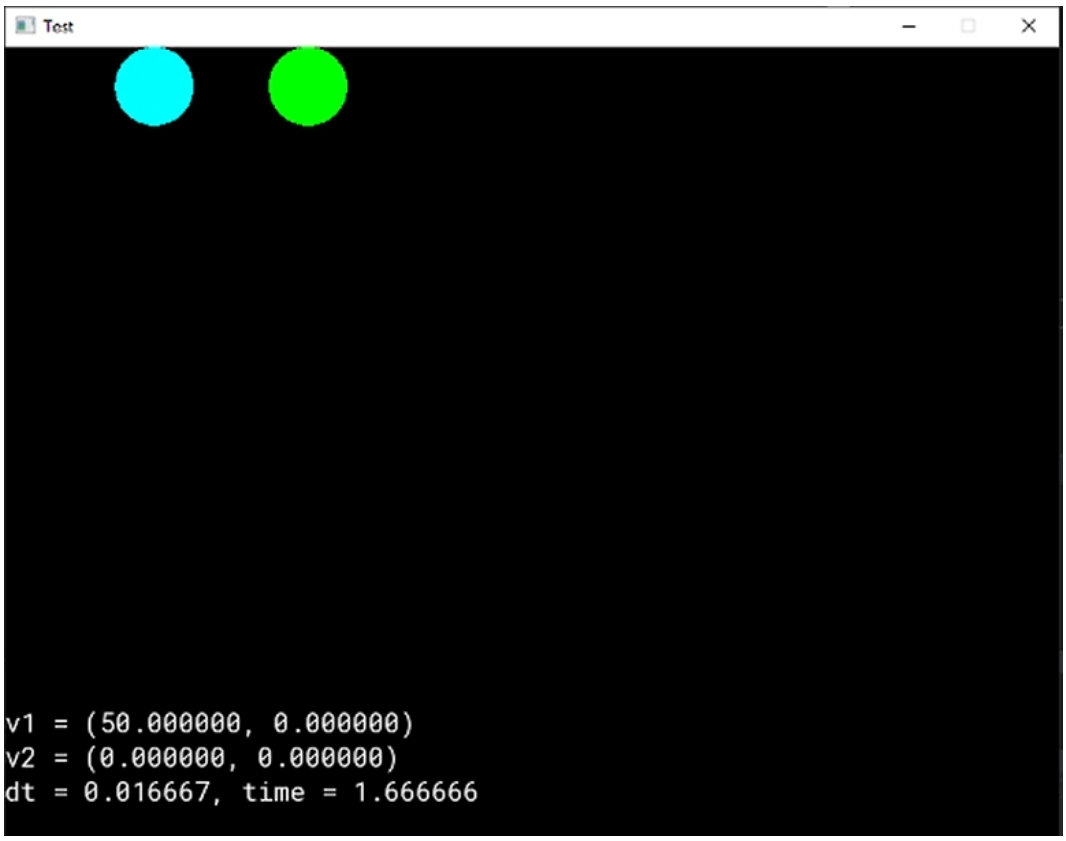
6.1.2 實作

```
void cir_collide(Circle& a, Circle& b) {        // using Hooke's law
    if (!if_collide(a, b).collide)
        return;

    int k = 100;
    float x = (a.get_radius() + b.get_radius()) - sfv::abs(a.get_pos() -
       b.get_pos());
    float abs_f = k * x;        // F = kx
    a.apply_force(abs_f * sfv::unitv(a.get_pos() - b.get_pos()));
    b.apply_force(abs_f * sfv::unitv(b.get_pos() - a.get_pos()));
}
```

6.1.3 成果

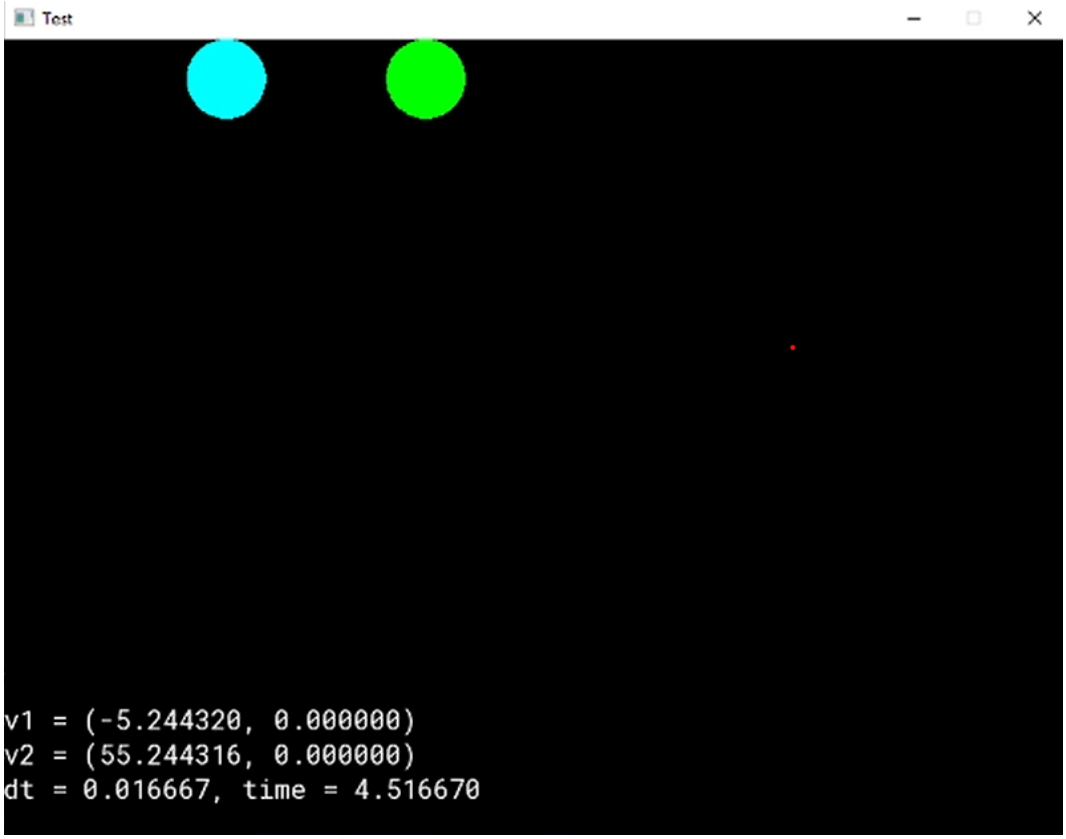
使用彈簧法模擬一維完全彈性碰撞 (每秒 60 幀，彈力常數 $k = 100$，兩球質量相等)，初始狀態如下圖：


Test
v1 = (50.000000, 0.000000)
v2 = (0.000000, 0.000000)
dt = 0.016667, time = 1.666666

碰撞中：

Test
v1 = (24.728367, 0.000000)
v2 = (25.271635, 0.000000)
dt = 0.016667, time = 2.916665

碰撞結果如下圖：



可以看到符合動量守恆，但卻不符合能量守恆，且誤差並不算小。

6.2 線性碰撞的改進：衝量法

彈簧法容易失真的原因在於：理論中的剛體碰撞應該僅有一瞬間，彈簧法的模擬卻會讓碰撞持續一段時間。這倒無妨，關鍵是在電腦模擬中不可能將時間無限細分，誤差便會隨著碰撞時間的拉長而增加。

衝量法可以很好的解決這個問題。此方法會在碰撞瞬間對兩物體施加同值反向的衝量，這滿足了動量守恆。因此衝量法的關鍵就在求出這個衝量的大小 j (方向是碰撞法向量)。

這邊僅作線性碰撞的推導，先不引入物體的旋轉，故先僅考慮圓。引入旋轉的碰撞，會在下一點詳述。

6.2.1 理論推導

給定兩圓心位置分別為 C_1, C_2，半徑 r_1, r_2，質量 m_1, m_2，碰撞前的速度為 $\vec{v}_1, \vec{v}_2$，碰撞後則為 $\vec{v_1}', \vec{v_2}'$，。碰撞法向量為 $\vec{n}$，彈力常數為 e。

首先列出三個式子：

$$\begin{cases} (\vec{v_1}' - \vec{v_2}') \cdot \vec{n} = -e(\vec{v}_1 - \vec{v}_2) \cdot \vec{n} \\ \\ \vec{v_1}' = \vec{v}_1 + \dfrac{j}{m_1}\vec{n} \\ \\ \vec{v_2}' = \vec{v}_2 - \dfrac{j}{m_2}\vec{n} \end{cases}$$

解聯立方程式即可求出 j。這邊省略繁瑣的代數推導，直接寫出結果。

$$j = \frac{-(1+e)(\vec{v_1}' - \vec{v_2}') \cdot \vec{n}}{(\frac{1}{m_1} + \frac{1}{m_2})\vec{n} \cdot \vec{n}}$$

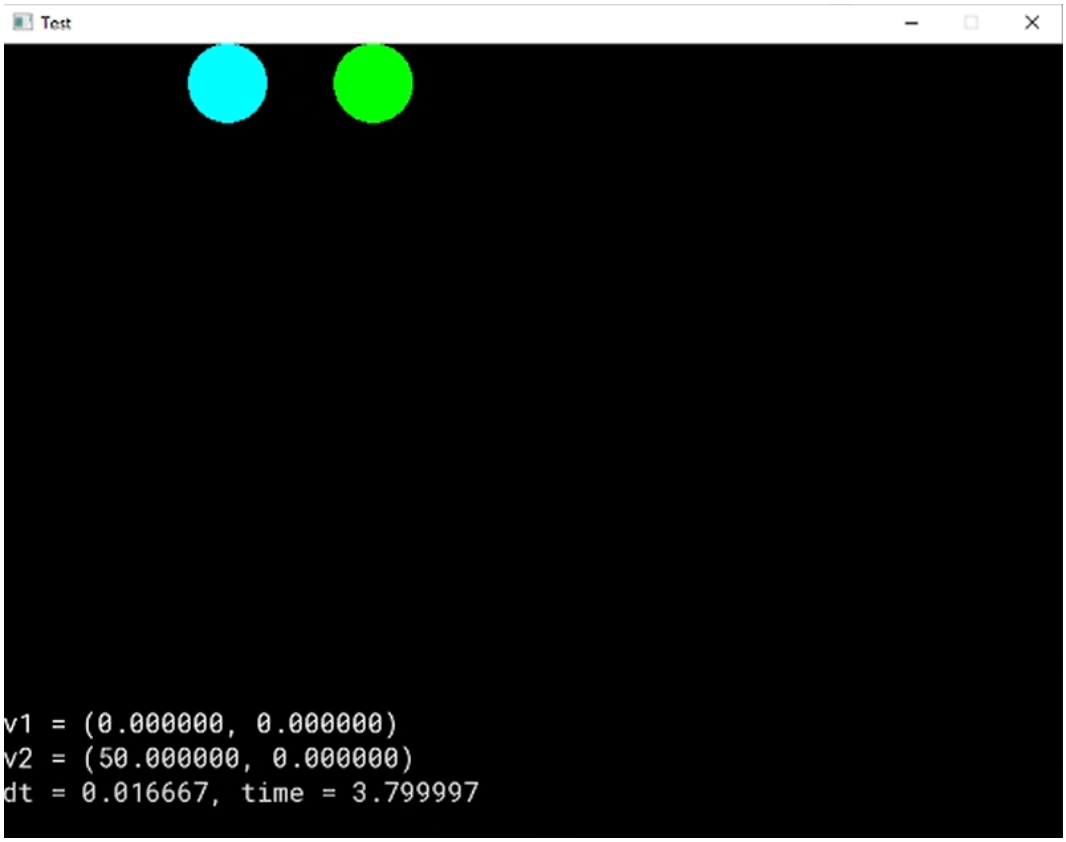
6.2.2 實作

```
void collide(Circle& a, Circle& b) {
    sf::Vector2f normal = b.get_pos() - a.get_pos();
    float e = a.get_e() * b.get_e();    // or min(a.e, b.e)
    float j = sfv::dot(-(1.f + e) * (a.get_v() - b.get_v()), normal) /
        sfv::dot(normal, normal * (a.get_rec_mass() + b.get_rec_mass()));
    a.set_v(a.get_v() + normal * (j / a.get_mass()));
    b.set_v(b.get_v() - normal * (j / b.get_mass()));
}
```

6.2.3 成果

使用衝量法模擬一維完全彈性碰撞 (每秒 60 幀，彈力常數 $k = 100$，兩球質量相等)，初始狀態如下圖：

碰撞結果如下圖：



完美符合理論結果，動量、能量皆守恆。

6.3 引入旋轉，大功告成：衝量法

上一點的方法已經可以很好的模擬線性碰撞，就差引入旋轉了。在引入旋轉後，碰撞公式就變得可以通用在圓及多邊形上，因此函式的參數型別可以直接寫成 `Shape`。

6.3.1 理論推導

在考慮旋轉的情況下，碰撞時的相對速度就不是質心速度相減那麼簡單了，而是接觸點上的質點速度相減。因此需先能求出一物體上特定位置質點的速度。有一物體速度 $\vec{v}$、角速度 ω，其上有一質點 P，質心到該質點的向量為 $\vec{r} = (r_x, r_y)$。當 ω 為正時物體逆時針旋轉，因此定義 $\vec{r}_\perp = (-r_y, r_x)$。求出該質點速度 $\vec{v}_P$ 為：

$$\vec{v}_P = \vec{v} + \omega\vec{r}_\perp$$

綜上所述，碰撞前後接處點相對速度分別為：

$$\vec{v}_{12} = (\vec{v}_1 + \omega_1\overrightarrow{r_{1\perp}}) - (\vec{v}_2 + \omega_2\overrightarrow{r_{2\perp}})$$

$$\vec{v}_{12}{}' = (\vec{v}_1{}' + \omega_1'\overrightarrow{r_{1\perp}}) - (\vec{v}_2{}' + \omega_2'\overrightarrow{r_{2\perp}})$$

至此，搭配力矩公式等，我們可以列出新的式子 (其中 I 為轉動慣量)：

$$\begin{cases} \vec{v_{12}}' \cdot \vec{n} = -e(\vec{v}_{12} \cdot \vec{n}) \\ \vec{v_1}' = \vec{v}_1 + \dfrac{j}{m_1}\vec{n} \\ \omega_1' = \omega_1 + \dfrac{\overrightarrow{r_{1\perp}} \cdot j\vec{n}}{I_1} \\ \vec{v_2}' = \vec{v}_2 - \dfrac{j}{m_2}\vec{n} \\ \omega_2' = \omega_2 - \dfrac{\overrightarrow{r_{2\perp}} \cdot j\vec{n}}{I_2} \end{cases}$$

再解聯立方程式得 j：

$$j = \frac{-(1+e)\vec{v}_{12} \cdot \vec{n}}{(\frac{1}{m_1} + \frac{1}{m_2})\vec{n} \cdot \vec{n} + \frac{(\overrightarrow{r_{1\perp}} \cdot \vec{n})^2}{I_1} + \frac{(\overrightarrow{r_{2\perp}} \cdot \vec{n})^2}{I_2}}$$

6.3.2 實作

在實作中須注意一點，當有超過一個接觸點時，須將衝量除以接觸點數目。另外，須將所有衝量計算完畢後，再對物體施以衝量，以免反過來用碰撞後的數據計算碰撞。

```
void collide(Shape& a, Shape& b, Collision_data cd) {
    if (!cd.collide)
        return;

    // delta vars
    sf::Vector2f del_va = sf::Vector2f(0, 0), del_vb = sf::Vector2f(0, 0)
        ;
    float del_omega_a = 0, del_omega_b = 0;

    sf::Vector2f n = cd.normal[0];
    for (int i = 0; i < cd.contact_point_count; i++) {

        float e = a.get_e() * b.get_e();        // or min(a.e, b.e)
        sf::Vector2f ra = sfv::normalv(cd.contact_points[i] - a.get_pos()
            );
        sf::Vector2f rb = sfv::normalv(cd.contact_points[i] - b.get_pos()
            );
        sf::Vector2f vap = a.get_v() + a.get_omega() * ra;
        sf::Vector2f vbp = b.get_v() + b.get_omega() * rb;
        float rec_Ia = a.get_rec_inertia();
        float rec_Ib = b.get_rec_inertia();

        float j_numerator = -(1.f + e) * sfv::dot((vap - vbp), n);
```

```
        float j_denominator = sfv::dot(n, n * (a.get_rec_mass() + b.get_rec_mass())) + sfv::square(sfv::dot(ra, n)) * rec_Ia + sfv::square(sfv::dot(rb, n)) * rec_Ib;
        float j = j_numerator / j_denominator;

        j /= cd.contact_point_count;

        del_va += n * (j * a.get_rec_mass());
        del_omega_a +=  sfv::dot(ra, j * n) * rec_Ia;
        del_vb += n * (-j * b.get_rec_mass());
        del_omega_b += sfv::dot(rb, -j * n) * rec_Ib;
    }

    a.set_v(a.get_v() + del_va);
    a.set_omega(a.get_omega() + del_omega_a);
    b.set_v(b.get_v() + del_vb);
    b.set_omega(b.get_omega() + del_omega_b);
}
```

7 問題解決

7.1 穿模

剛寫完碰撞反應、正等著測試慶功時，卻發現物體穿過牆壁而出的情況不時發生。經過更詳細的觀察與測試，當物體速度較快或幀數較低時，物體容易因碰撞時有些許重疊而導致碰撞反應出錯。解決方法是在碰撞時除了對物體施加衝量，還要移開重疊的部分。

在6.3.2的最後加入以下程式碼可解決該問題：

```
sf::Vector2f overlap = sfv::unitv(n) * cd.depth * (float)0.9;
if (sfv::dot(overlap, a.get_pos() - b.get_pos()) < 0)
        overlap = -overlap;


if (a.get_immovable())
        b.move(-overlap);
else if (b.get_immovable())
        a.move( overlap);
else {
        a.move( overlap * b.get_mass() / (a.get_mass() + b.get_mass()));
        b.move(-overlap * a.get_mass() / (a.get_mass() + b.get_mass()));
}
```

7.2 能量不守恆

在測試時，時常彈力常數明明設定為 1，卻能明顯見到系統總力學能不停在下降。仔細觀察後發現，物體與牆壁發生碰撞時的行為並不真實。

會導致此種錯誤的主要原因，是不可動物體 (如牆壁) 的質量及轉動慣量設定錯誤。對於該種物體，理論上用無限大的質量及轉動慣量可以正確模擬，但礙於 C++ 中似乎無法表示

無限大，只能**在使用到質量或轉動慣量時盡可能改以使用其倒數**，此時帶入 0 便得以正確模擬。

8 未來展望

8.1 新增使用者介面

目前要進行物體狀態 (數量、位置、形狀、初速度等) 及參數 (重力強度、幀數等) 的調整均只能透過調整程式碼再重新編譯的方式進行。若能新增使用者介面，並能在執行時進行上述操作，實用性和方便性都將大幅提升，不了解程式設計的使用者也能輕鬆使用。

8.2 碰撞偵測的效能改進

目前偵測的方式是：對於所有的物體，檢查有沒有與其他所有物體碰撞。若有 N 個物體，則時間複雜度為 $O(N^2k)$，其中 k 為檢查兩物體是否碰撞的時間複雜度。這導致在物體較多的時候容易發生卡頓狀況。

可能的解決方法：引入四叉樹 (QuadTree) 演算法，將空間分割，在物體多時可大幅減少檢查的次數。

8.3 新增摩擦力

本專題是在假設物體表面皆為光滑的情況下進行。話雖如此，引入摩擦力應不會大幅變更基本架構或核心，僅會在碰撞時增加一個因素，因此有望未來能在作品中新增摩擦力。

9 心得與收穫

在這次的專題製作過程，我的收穫與提升可以分為硬實力和軟實力兩部分。

首先在硬實力的部分，我在大量撰寫程式碼的過程中熟悉了基本工具的使用，如使用 Library 的方法、善用 STL 及 iterator 等能大幅提升程式碼可讀性及撰寫速度的工具，也對寫程式這件事情更加駕輕就熟。再者，這是我首次嘗試製作專案，在命名風格、專案架構、常見慣例等方面都從毫無頭緒到逐漸清晰。尤其在程式碼的量愈發增加之後，維持好的註解習慣和程式碼品質就變得至關重要，要是可讀性不佳會嚴重影響工作效率。雖然在一些細部的邏輯和簡潔度方面尚有改善空間，但我已打下基礎，相向再透過更多的學習及訓練能夠更向上一層樓。

至於軟實力，主要在於除錯能力的增長和心態的轉變。從前我常在遇到瓶頸或找不出錯誤時鬼打牆或選擇放棄，然而在這次專題製作的經驗中，我學會正面困難，並嘗試有系統的解決。先冷靜下來，使用 Debugger 找出錯誤發生的點，也可自行模擬程式的運行，看哪個步驟和預想的不同。同時，確保每部分都正確地完成再進行下一步，而不是忽視可能存在的問題、得過且過，否則易留下後患，日後得花上更多時間來解決。正所謂地基打得穩，樓才蓋得高。

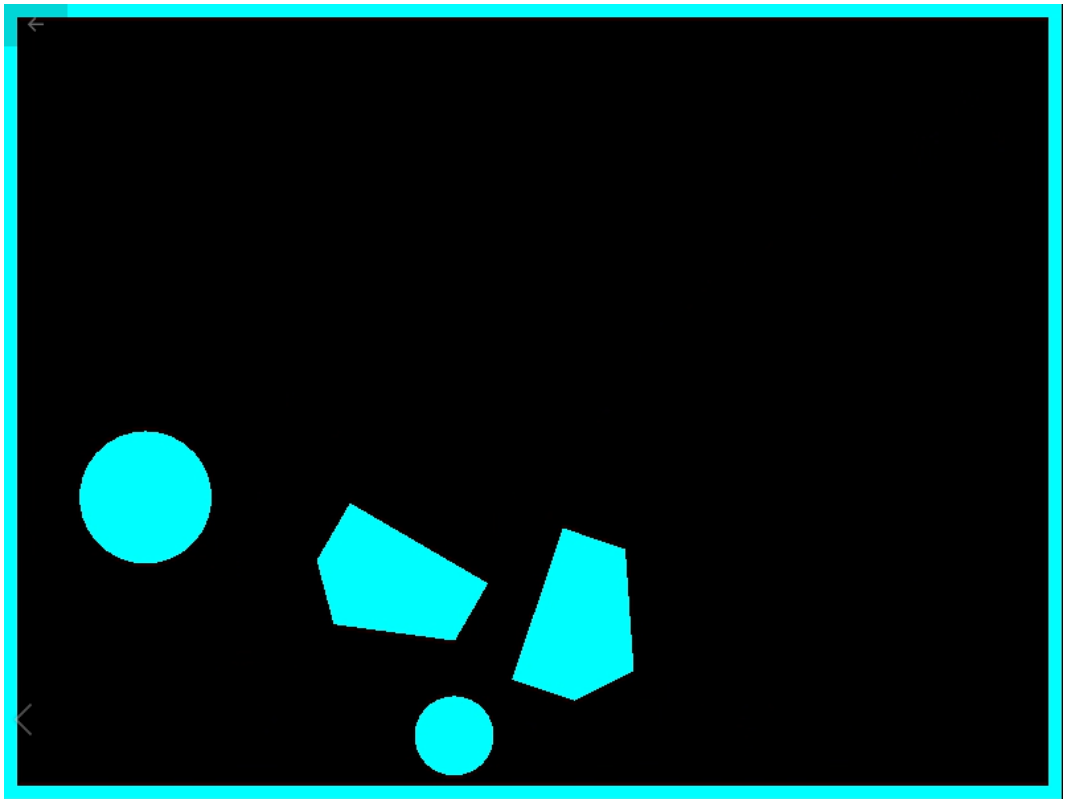
另外，在專題的起步階段，我選擇先大量搜尋資料、進行鳥瞰，先大量略讀相關資料建立印象，而非一開始便埋頭苦幹。這種學習方法在我遇到困難時給了我很大幫助，事前建立的印象會使我大略知道該往哪個方向解決問題，不會毫無頭緒，也能在一些關鍵時刻給我靈感，甚至讓我提前意識到可能存在的問題並斬草除根。在環境變化快速的年代，此種後設認知 (Metacognition) 的能力更顯重要。我在學習專題所需的知識時也不停嘗試著有意識地控制我的學習策略，我相信這是能幫助我適應環境、跟上時代的重要能力。

10 附錄：作品連結

10.1 Github 連結

https://github.com/House0521/Physics_engine.git

10.2 成果截圖



10.3 測試影片

https://www.youtube.com/watch?v=Y8MliJ0AhoM